

POROČILO - RSO SEMINAR 2025/2026

Osnovni podatki

- **Naslov projekta:** Predelava aplikacije Kamen-Škarje-Papir v Cloud Native Mikro Storitve
 - **Člana skupine:** Bernard Kučina, Filip Merkan
 - **Povezava do organizacije:** <https://github.com/RSO-skupina-0>
 - **Povezava do aplikacije:** <http://kamen-skarje-papir.click>
-

Kratek opis projekta

Projekt predstavlja predelavo obstoječe monolitne aplikacije "Kamen-Škarje-Papir" ki je objavljena v [GitHub organizaciji](#) v sodobno cloud native arhitekturo z uporabo mikro storitev. Trenutna aplikacija, ki podpira dve različici igre (klasično KŠP in razširjeno KŠPOV z Ogenj/Voda), bo razdeljena na 4 neodvisnih mikro storitev, ki bodo omogočale boljšo skalabilnost, vzdrževanje in razširljivost. Rešitev bo rešila probleme monolitne arhitekture, kot so težko vzdrževanje, omejena skalabilnost in tesno povezanost komponent, ter zagotovila visoko dostopnost in odpornost na napake.

Ogrodje in razvojno okolje

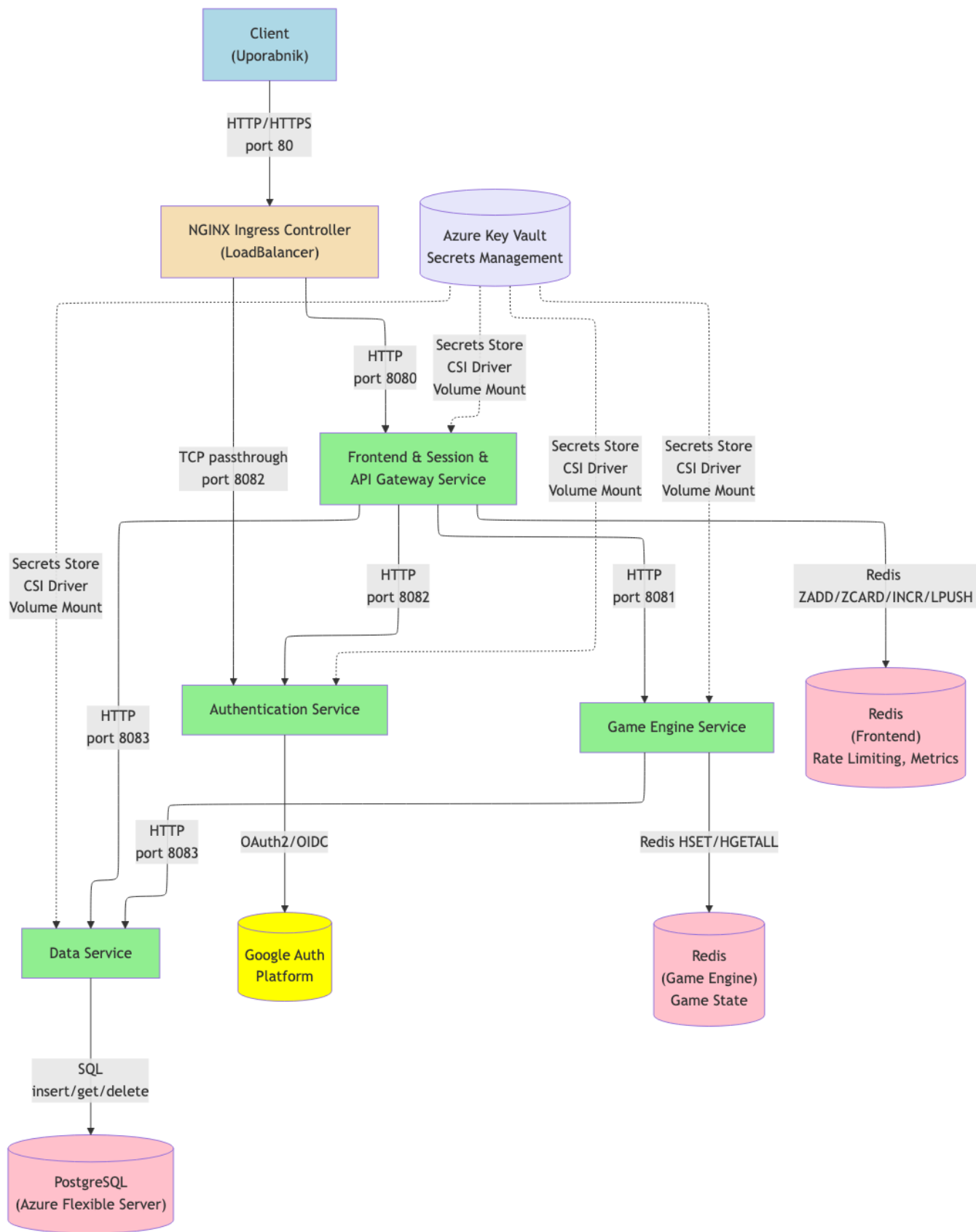
Tehnologije in ogrodja:

- **Backend:** Python 3.11+, spletni strežnik Gunicorn in Bottle spletno ogrodje
- **Baze podatkov:** PostgreSQL (trajno shranjevanje), Redis (seje)
- **Vsebniki:** Docker, Docker Compose
- **Orkestracija:** Kubernetes
- **Komunikacija:** REST API
- **Avtentikacija:** JWT, OAuth 2.0 integracija
- **CI/CD:** GitHub Actions
- **API Gateway:** NGINX

Razvojno okolje

- **IDE:** Visual Studio Code
 - **Upravljanje različic:** Git, GitHub
 - **Testiranje API-jev:** Swagger
 - **Dokumentacija:** Swagger (OpenAPI 3.0), Markdown
-

Shema arhitekture



Seznam funkcionalnosti mikrostoritev

1. Authentication Service

Funkcionalnosti: integracija OAuth2/OIDC (Google) in validacija ID žetonov.

Glavne poti:

- **GET /auth/google/login** – preusmeritev uporabnika na *Google Auth Platform*
- **GET /auth/google/callback** – obdelava povratnega klica iz *Google Auth Platform*
- **GET /auth/redeem** – vrne podatke o prijavljenem uporabniku na podlagi vstopnice

2. Game Engine Service

Funkcionalnosti: implementacija igralne logike za KŠP igr KŠPOV, shranjevanje stanja v trenutne igre na Redis.

Glavne poti:

- **POST /game/ksp/nova** – inicializira novo igro *KŠP*
- **GET /game/ksp/state** – vrne trenutno stanje igre *KŠP*
- **POST /game/ksp/poteza** – izvede potezo in izračuna novo stanje igre *KŠP*
- **POST /game/kspov/nova** – inicializira novo igro *KŠPOV*
- **GET /game/kspov/state** – vrne trenutno stanje igre *KŠPOV*
- **POST /game/kspov/poteza** – izvede potezo in izračuna novo stanje igre *KŠPOV*

4. Data Service

Funkcionalnosti: shranjevanje zaključenih iger (PostgreSQL), pregled zgodovine iger, brisanje zgodovine.

Glavne poti:

- **POST /data/ksp/insert** – zapiše končni rezultat igre *KŠP* v podatkovno bazo
- **GET /data/ksp/get/id** – pridobi ustrezen identifikator igre
- **GET /data/ksp/history** – vrne zgodovino iger uporabnika
- **POST /data/ksp/delete** – izbriše zgodovino iger uporabnika
- **POST /data/kspov/insert** – zapiše končni rezultat igre *KŠPOV* v podatkovno bazo
- **GET /data/kspov/get/id** – pridobi ustrezen identifikator igre
- **GET /data/kspov/history** – vrne zgodovino iger uporabnika
- **POST /data/kspov/delete** – izbriše zgodovino iger uporabnika

5. Frontend, Session, API Gateway Service

Funkcionalnosti: usmerjanje zahtev na mikrororitve, prikaz uporabniškega vmesnika (HTML/CSS/JS), upravljanje sej (piškotki), omejevanje hitrosti zahtevkov (rate limiting), porazdeljevanje bremena med mikrosotiravmi (load balancing).

Glavne poti:

- **GET /** – prikaz strani za prijavo uporabnika
- **PUT /frontend/login** – prijava uporabnika (*Gost* ali *Uporabnik*) in nastavev piškotkov
- **GET /auth/finalize** – klic mikrororitve *Authentication Service* za dokončanje avtentikacije
- **GET /igra** – prikaz začetnega menija za igranje iger *KŠP* in *KŠPOV*
- **POST /ksp** — pridobi podatke o uporabnikovi izbiri (kamen, škarje ali papir)
- **GET /ksp** — iz piškotkov prebere podatke o uporabniku, pridobi stanje igre *KŠP*
- **GET /nova_igra_ksp** — inicializacija nove igre *KŠP*
- **GET /zgodovina_ksp** — pridobi podatke o zgodovini iger
- **DELETE /brisi_ksp** — izbriše zgodovino iger *KŠP* (klic mikrororitve *Data Service*)
- **POST /kspov** — pridobi podatke o uporabnikovi izbiri (kamen, škarje, papir, ogenj ali voda)
- **GET /kspov** — iz piškotkov prebere podatke o uporabniku, pridobi stanje igre *KŠPOV*
- **GET /nova_igra_kspov** — inicializacija nove igre *KŠPOV*
- **GET /zgodovina_kspov** — pridobi podatke o zgodovini iger *KŠPOV*
- **DELETE /brisi_kspov** — izbriše zgodovino iger *KŠPOV* (klic mikrororitve *Data Service*)

Primeri uporabe

Osnovni primeri uporabe:

1. Prijava naročnika

- Naročnik dostopa do aplikacije prek domene <http://kamen-skarje-papir.click>.
- Naročnik klikne gumb »**Prijava Google**«.
- Sistem naročnika preusmeri na OAuth ponudnika.
- OAuth ponudnik po uspešni prijavi vrne avtentikacijsko kodo.
- Naročnik je preusmerjen na začetni meni aplikacije.

2. Prijava uporabnika

- Uporabnik dostopa do aplikacije prek domene <http://kamen-skarje-papir.click>.
- Uporabnik vnese svoje uporabniško ime.
- Uporabnik klikne gumb »**Prijava**«.

3. Prijava gosta

- Gost dostopa do aplikacije prek domene <http://kamen-skarje-papir.click>.
- Gost klikne gumb »Gost«.

4. Nova igra KŠP/KŠPOV

- Uporabnik/gost/naročnik klikne gumb »KŠP« / »KŠPOV«.
- Prikaže se igralni vmesnik za igro KŠP.
- Ob tem se prek klica mikrororitve **Game Engine Service** ustvari nova instanca igre KŠP.

5. Igranje poteze KŠP

- Uporabnik/gost/naročnik izbere možnost (**kamen/škarje/papir**).
- Mikrororitev **Game Engine Service** izračuna rezultat poteze.
- V igralnem vmesniku se prikaže posodobljen rezultat.

6. Igranje poteze KŠPOV

- Uporabnik/gost/naročnik izbere možnost (**kamen/škarje/papir/ogenj/voda**).
- Mikrororitev **Game Engine Service** izračuna rezultat poteze.
- V igralnem vmesniku se prikaže posodobljen rezultat.

7. Pregled zgodovine KŠP/KŠPOV

- Naročnik zahteva pregled zgodovine iger KŠP/KŠPOV.
- Mikrororitev **Data Service** vrne shranjene rezultate iger KŠP/KŠPOV.
- Prikaže se zgodovina iger KŠP.

8. Brisanje zgodovine

- Naročnik zahteva brisanje zgodovine iger.
- Mikrororitev **Data Service** izbriše podatke iz baze **PostgreSQL**.
- Prikaže se prazna zgodovina iger.

Kompleksnejši primer uporabe:

Tok:

1. Naročnik dostopa do aplikacije prek domene <http://kamen-skarje-papir.click>.
2. Naročnik klikne gumb »Prijava z Googlom« in se uspešno avtenticira.
3. Naročnik klikne gumb »KŠPOV«; mikrororitev **Game Engine Service** ustvari novo instanco igre.
4. Naročnik odigra igro, igralni vmesnik pa prikaže končni rezultat.
5. Mikrororitev **Data Service** shrani rezultat v bazo **PostgreSQL**.
6. Naročnik klikne gumb »Zgodovina«.
7. Mikrororitev **Data Service** pridobi podatke o zgodovini iger KŠPOV.
8. Naročnik zahteva brisanje zgodovine.
9. Prikaže se prazna zgodovina iger.

Seznam opravljenih/vključenih osnovnih in dodatnih projektnih zahtev

Postavka	Opis rešitve
----------	--------------

Postavka	Opis rešitve
Repozitorij	Repozitorij je vsebovan v GitHub organizaciji RSO-skupina-03 . Repozitorij vsebuje: izvorno kodo, Dockerfile-e, Kubernetes manifeste (Deployment, Service, ConfigMap), CI/CD pipeline definicije (GitHub Actions), Azure deployment skripte in celotno dokumentacijo. Uporabljena je strategija razvejitve z master branch-om kot glavno produkcijsko vejo.
Mikrostoritve in »cloud-native« aplikacija	Aplikacija je razdeljena na štiri mikrostoritve, razvite v Python 3.11+ z ogrodjem Bottle in WSGI strežnikom Gunicorn. Mikrostoritve so: Authentication Service, Game Engine Service, Data Service ter Frontend, Session & API Gateway Service . Vsaka mikrostoritev ima svoj Dockerfile ter Kubernetes manifesta Deployment in Service . Vse storitve so nameščene v okolju Azure Kubernetes Service (AKS) . Za trajno shranjevanje se uporablja baza PostgreSQL (Azure Flexible Server) z ločenimi tabelami za igre KŠP in KŠPOV .
Dokumentacija	Repozitorij vsebuje dokumentacijo v formatu Markdown: README.md z navodili za lokalno in oblačno namestitev ter opisom arhitekture; dokumentacija.md s podrobnim opisom uporabljenih tehnologij in mikrostoritev; ter porocilo.md z opisom projekta, seznamom funkcionalnosti, primeri uporabe in izpolnjenimi zahtevami.
Dokumentacija API	Vse mikrostoritve izpostavljajo interaktivni Swagger UI na poti /docs z OpenAPI 3.0 specifikacijo. Dokumentacija vključuje podrobnosti o vseh glavnih poteh, formatih zahtevkov in odgovorov ter HTTP statusnih kodah. Swagger UI omogoča interaktivno testiranje API-jev neposredno v brskalniku.
Cevovod CI/CD	GitHub Actions pipeline avtomatizira celoten razvojni cikel z dvema workflow datotekama: ci-cd.yml za CI korake (checkout kode, nastavitve Python okolja, testiranje z pytest, gradnja Docker slik za vse 4 storitve, push v Docker Hub) in deploy.yml za CD korake (Azure login z OIDC, pridobitev AKS credentials, posodabljanje SecretProviderClass z dinamičnimi vrednostmi, nameščanje Kubernetes manifestov, restart deployment-ov, čakanje na rollout status). Pipeline se sproži ob push-u v master branch ali ročno prek workflow_dispatch . Vse Docker slike so shranjene v Docker Hub (filipmerkan/ksp-*) in se avtomatsko posodobijo ob vsaki spremembi.
Helm charts	
Namestitev v oblak	Aplikacija je nameščena v Microsoft Azure z uporabo Azure Kubernetes Service (AKS) v regiji Sweden Central. Kubernetes gruča ksp-aks-cluster-microservices je javno dostopna z eno node skupino (Standard_B2s VM size) in uporablja managed identity za avtentikacijo. Aplikacija je javno dostopna prek domene kamen-skarje-papir.click z NGINX Ingress Controller LoadBalancer IP. Vse mikrostoritve so nameščene v AKS z Kubernetes manifesti (Deployment, Service), Redis instance za Frontend in Game Engine so nameščene v gruča, PostgreSQL baza podatkov je na Azure Flexible Server, Azure Key Vault se uporablja za upravljanje skrivnosti. Celotna infrastruktura je konfigurirana z Azure CLI skriptami in GitHub Actions CI/CD pipeline-om.

Postavka	Opis rešitve
Preverjanje zdravja	Vse mikrostoritve izpostavljajo dva endpointa: <code>/health</code> (liveness probe), ki preveri, ali proces teče, in vrne HTTP 200, ter <code>/ready</code> (readiness probe), ki preveri razpoložljivost odvisnosti in vrne HTTP 200 ali 503. Kubernetes Deployment manifesti vključujejo konfiguraciji <code>livenessProbe</code> in <code>readinessProbe</code> z HTTP GET zahtevki na navedena endpointa, intervalom 10 sekund, časovno omejitev 5 sekund (<code>timeoutSeconds</code>) in <code>failureThreshold: 3</code> . Ko <code>readinessProbe</code> ne uspe, Kubernetes odstrani pod iz nabora endpointov storitve (<code>Service</code>); ko <code>livenessProbe</code> ne uspe, Kubernetes pod ponovno zažene.
Zbiranje metrik	Frontend service zbira metrike o zahtevkih z middleware funkcijo, ki beleži vsak zahtevek v Redis z uporabo Redis ukazov (INCR za števce, LPUSH za log zapise, LTRIM za ohranjanje zadnjih 1000 vnosov). Metrike vključujejo: skupno število zahtevkov (<code>gateway:metrics:total</code>), število zahtevkov po endpointih (<code>gateway:metrics:endpoint:{endpoint}</code>), statusne kode (<code>gateway:metrics:status:{status}</code>), čase odziva (<code>gateway:metrics:response_times</code>) ter log zapise z IP naslovi, časovnimi žigi in endpointi (<code>gateway:metrics:logs</code>).
Izolacija in toleranca napak	Sistem implementira odpornost na napake na več ravneh: preverjanja zdravja (liveness/readiness probe) omogočajo Kubernetesu samodejno zamenjavo nezdravih podov. Retry logika v Frontend service-u ob napakah poskuša ponovno izvesti klice do zalednih storitev. Vzorec circuit breaker preprečuje kaskadne napake z začasnim blokiranjem zahtevkov do nezdravih storitev. Shranjevanje stanja v Redis omogoča hitro obnovitev po napaki brez izgube stanja igre. Kubernetes pa zagotavlja visoko razpoložljivost z več replikami storitev.
Uporavljanje s konfiguracijo	
Grafični vmesnik	Grafični vmesnik je sestavljen iz šestih podstrani, ki vključujejo prijavo, izbiro igre, igralno konzolo ter prikaz zgodovine iger. Razvit je v spletnem ogrodju Bottle. Z ostalimi mikrostoritvami komunicira prek REST API-ja.
Terraform	
API Gateway	Frontend service deluje kot API Gateway z usmerjanjem zahtevkov na zaledne storitve (Game Engine na port 8081, Authentication na port 8082, Data Service na port 8083) prek HTTP klicev. Gateway upravlja seje z uporabo piškotkov (skrivnost za podpis je shranjena v Azure Key Vault), implementira rate limiting z Redis (omejitev zahtevkov po IP naslovu in endpoint-u), zbira metrike in izvaja logging. Gateway je integriran z NGINX Ingress Controller za zunanji dostop prek domene <code>kamen-skarje-papir.click</code> .
Ingress Controller	NGINX Ingress Controller je nameščen v Kubernetes namespace <code>ingress-nginx</code> z uporabo Helm chart-a. Konfiguriran je z Ingress manifestom za HTTP routing (port 80) do Frontend storitve na poti <code>/</code> ter z nastavitvami za zunanji dostop prek domene <code>kamen-skarje-papir.click</code> (DNS A zapis kaže na LoadBalancer IP). Ingress Controller upravlja load balancing, health checks in (po potrebi) TLS/SSL terminacijo.

Postavka	Opis rešitve
IAM, OAuth2, OIDC	Authentication Service implementira protokol OAuth 2.0 / OIDC z Google Auth Platform kot zunanjim IAM ponudnikom. Sistem podpira tri tipe uporabnikov, pri čemer se naročnik prijavi prek Google OAuth2/OIDC. Avtentikacija uporablja enkratne vstopnice (one-time tickets) z omejenim časom veljavnosti (TTL 60 s) za varno posredovanje podatkov med storitvami, brez neposrednega posredovanja žetonov brskalniku. OAuth poverilnice (client ID/secret) so shranjene v Azure Key Vault in dostopne prek Secrets Store CSI Driver.